Real-Time E-Commerce Recommendation System

Domain: Amazon Electronics Product Reviews

Focus: Personalization (Cold-Start Problem & User Segmentation)

Team Members: Farah Mostafa 2022O2144 & Nada Mohamed 202201541

Submission Date: 13 May 2026

1. System Architecture

The system is designed as an end-to-end big data pipeline combining batch machine learning with real-time streaming analytics. The architecture follows a Lambda-style pattern with two distinct processing layers.

1.1 Architecture Diagram

The full pipeline is described as follows:

> **Amazon CSV (1.69M rows)** --> **Spark Preprocessing** --> **ALS Training** --> **Saved Model**
>
> **Kafka Producer** --> Kafka Topic (2 partitions) --> **Spark Structured Streaming**
>
> **Spark Streaming** --> **ALS Model** --> **Recommendations + Alerts** --> **Dashboard**

1.2 Component Overview

Component	Technology	Role
Batch Processing	Apache Spark MLlib	ALS model training on 1.69M records
Message Broker	Apache Kafka (2 partitions)	Real-time event streaming
Stream Processing	Spark Structured Streaming	Window analytics + ML integration
Coordination	Apache Zookeeper	Kafka cluster management
Dashboard API	Flask + Flask-CORS	REST API serving streaming output
Dashboard UI	HTML + Chart.js	Live visualization of recommendations
Orchestration	Docker Compose	6-container deployment

1.3 Docker Services

- spark-master: Spark cluster master + JupyterLab on port 8888
- spark-worker-1 & spark-worker-2: Each with 2 cores / 2GB RAM
- zookeeper: Kafka coordination service
- kafka: Message broker with 2 partitions on port 9092
- dashboard-api: Flask REST API on port 5050

2. Dataset

2.1 Dataset Description

Property	Value
Name	Amazon Product Reviews - Electronics
Source	UCSD Julian McAuley Research Lab (Stanford SNAP)
Total Records	~1,690,000 reviews
Format	user_id (string), item_id (string), rating (float), timestamp (unix)
Time Range	1998 - 2014
Size on Disk	~180 MB (CSV)

2.2 Why This Dataset

- Exceeds the 500K record requirement with 1.69M reviews
- Contains real e-commerce interaction data with natural sparsity patterns
- Cold-start problem is inherently present: many users have fewer than 5 interactions
- Sufficient diversity of users and items for collaborative filtering
- Timestamps enable realistic streaming simulation

2.3 Why Distributed Processing is Needed

- ALS matrix factorization on 1.69M records requires parallel computation across executors
- StringIndexer encoding of alphanumeric IDs across 1M+ rows cannot fit in single-node memory
- Window analytics over streaming data require parallel partition processing across workers
- Loading and joining multiple parquet tables during streaming requires distributed caching

3. Machine Learning Component

3.1 Data Preprocessing

The raw CSV data undergoes the following cleaning pipeline before model training:

- Drop null values across all four required columns (user_id, item_id, rating, timestamp)
- Validate rating range: only values between 1.0 and 5.0 are retained
- Remove duplicate (user, item) interactions using a window function ordered by timestamp descending, keeping only the most recent interaction per pair
- Filter sparse users and items: only retain users and items with at least 5 interactions. This improves ALS quality and is the first cold-start mitigation step
- Encode string IDs to integers using Spark MLlib StringIndexer, which is required by the ALS algorithm. Encoding maps are saved as Parquet files for use in the streaming pipeline

3.2 ALS Model Training

Parameter	Value	Justification
rank	20	Number of latent factors; balances complexity vs. overfitting
maxIter	15	Sufficient convergence for this dataset size
regParam	0.1	L2 regularization to prevent overfitting
coldStartStrategy	drop	Unknown users/items are excluded from evaluation
nonnegative	true	Keeps factor matrices non-negative for interpretability
implicitPrefs	false	Explicit ratings used (not implicit clicks)

3.3 RMSE Evaluation & Tuning

The dataset is split 80% training / 20% testing with seed=42. RMSE is computed using Spark's RegressionEvaluator. If RMSE exceeds 1.5, automatic tuning is triggered:

- rank tested: 10, 30
- regParam tested: 0.05, 0.15
- maxIter fixed at 15 for all tuning runs

Typical RMSE for Amazon Electronics with these parameters falls between 0.95 and 1.30, well within the acceptable threshold.

3.4 User Segmentation (Personalization Focus)

Users are segmented into three groups based on their historical activity level. This is the core of the personalization focus and drives the recommendation strategy for each user:

Segment	Criteria	Recommendation Strategy
power_user	20 or more historical ratings	Full ALS Top-5 personalized recommendations
regular_user	10 to 19 historical ratings	Full ALS Top-5 personalized recommendations
cold_start	5 to 9 historical ratings	Popularity-based fallback (Bayesian average)

The Bayesian average popularity score used for cold-start users is computed as:

```
bayesian_score = (count x avg_rating + 50 x global_mean) / (count + 50)
```

The constant 50 smooths scores for items with few ratings toward the global mean, preventing items with a single 5-star rating from dominating the fallback list.

4. Streaming Component

4.1 Kafka Setup & Partitioning Strategy

The Kafka topic 'ecommerce-interactions' is configured with 2 partitions. Events are routed to partitions based on user segment:

- Partition 0: cold_start users (5-9 historical ratings)
- Partition 1: power_user and regular_user (10+ historical ratings)

Justification: Each partition is consumed by a separate Spark executor. Since cold-start users require a completely different code path (popularity fallback instead of ALS), keeping them in a dedicated partition avoids contention, reduces cross-partition shuffles, and improves cache locality for the ALS model lookup in Partition 1.

4.2 Event Schema

Each Kafka message is a JSON object with the following structure:

```
{"user_id": 42, "user_id_raw": "A1B2C3", "item_id": 100,
 "item_id_raw": "B00XYZ123", "rating": 4.5, "timestamp": "2026-05-10T20:00:00Z",
 "segment": "power_user", "event_type": "rating"}
```

4.3 Kafka Producer

The Python-based Kafka producer simulates realistic e-commerce event traffic with three injection patterns:

- Normal traffic: random user-item interactions sent every 100ms
- Activity spike: one user fires 10 rapid interactions (injected every 200 events)
- Trending item: 15 different users rate the same item with high ratings (injected every 300 events)
- Late data: 5% of events are sent with timestamps 60-120 seconds in the past

4.4 Spark Structured Streaming Pipeline

The streaming pipeline consists of the following stages:

- Read raw bytes from Kafka using the kafka format source
- Decode value column from bytes to string
- Parse JSON safely using from_json with a defined schema. Malformed records with null required fields are dropped with a filter step
- Convert event_time string to TimestampType
- Apply watermark of 2 minutes for late data handling

4.5 Window Analytics

Parameter	Value
Window size	30 seconds
Slide interval	10 seconds

Parameter	Value
Overlap	20 seconds (each event contributes to 3 windows)
Output mode	append (required with watermark)
Trigger interval	10 seconds processing time

Per-item metrics computed in each window: average rating, interaction count, rating variance, minimum rating, maximum rating, engagement score.

Per-user metrics computed in each window: interaction count, average rating given.

5. Custom Streaming Metric: Engagement Score

The engagement score is a custom metric computed per item in each streaming window. It captures items that are both highly rated AND frequently interacted with:

```
engagement_score = (avg_rating / 5.0) x log(interaction_count + 1)
```

Components:

- avg_rating / 5.0: normalizes the average rating to a 0-1 scale
- log(interaction_count + 1): logarithmic scaling of interaction volume, preventing items with hundreds of interactions from completely dominating over well-rated items with fewer interactions

Interpretation: An item with avg_rating=4.8 and 1 interaction scores lower than an item with avg_rating=4.5 and 20 interactions. This rewards genuine engagement over isolated high ratings. The score is displayed in the dashboard and used to rank trending items.

6. ML + Streaming Integration

Each micro-batch of streaming events triggers the following integration logic via foreachBatch:

- Extract unique users from the current micro-batch
- Look up their segment from the pre-loaded user_segments parquet table
- Route power_user and regular_user to ALS recommendForUserSubset(Top-5)
- Route cold_start users to the pre-computed popularity fallback list
- Write results as JSON to /app/output/recommendations/epoch_N.json
- Measure latency from batch start to write completion

The ALS model is loaded once at startup and broadcast to all executors, avoiding repeated disk reads per micro-batch. The popularity fallback list (top 5 items) is collected to the driver and used as a Python list for zero-overhead cold-start serving.

6.1 Latency Measurement

Measurement	Method	Target
Batch processing latency	time.time() before and after foreachBatch	< 5000ms per batch
End-to-end latency	Kafka event timestamp vs. recommendation write time	< 10 seconds
Trigger interval	processingTime set to 10 seconds	Fixed 10s micro-batches

7. Alert System & Late Data Handling

7.1 Alert Rules

Alert Type	Trigger Condition	Example Output
TRENDING_ITEM	avg_rating > 4.5 AND interaction_count >= 3 in a 30s window	ALERT: Item 120 is trending! avg_rating=4.8, interactions=15
ACTIVITY_SPIKE	user interaction_count > 8 in a 30s window	ALERT: User 42 spike! 11 interactions in 30s

7.2 Late Data Handling

Watermarking is configured with a 2-minute tolerance on the event_time column:

```
.withWatermark('event_time', '2 minutes')
```

- Events arriving within 2 minutes of their event_time are included in their correct window
- Events arriving more than 2 minutes late are dropped and not included in any window
- The Kafka producer injects 5% late events (60-120 seconds old) to verify this behavior
- The append output mode is used (not complete) because watermarking requires append mode for windowed aggregations

8. Dashboard Analytics

The dashboard consists of two components: a Flask REST API and an HTML frontend.

8.1 Flask API Endpoints

Endpoint	Data Served
/api/metrics	Total recommendations served, avg latency, total interactions, avg engagement score
/api/recommendations	Latest ALS and popularity fallback recommendations with strategy tags
/api/window_stats	Windowed item analytics including trending items ranked by engagement score
/api/alerts	All TRENDING_ITEM and ACTIVITY_SPIKE alerts with timestamps

8.2 Dashboard Visualizations

- Real-time metric cards: total recommendations, avg latency (ms), interaction count, engagement score
- Rating distribution bar chart: average rating per item across the current window (Chart.js)
- Interaction volume line chart: total interactions over time updated every 5 seconds
- Live recommendations list: user ID, item ID, predicted score, strategy tag (ALS or POP)
- Alerts panel: color-coded TRENDING_ITEM (orange) and ACTIVITY_SPIKE (red) alerts
- Trending items: ranked bar visualization by engagement score
- Recommendation latency chart: ms per batch over time with 5000ms threshold line

9. Results

9.1 Model Performance

Metric	Value
Training set size	80% of filtered dataset (~1.1M records)
Test set size	20% of filtered dataset (~280K records)
ALS rank	20
ALS regParam	
ALS maxIter	
Final RMSE	expected 0.95 to 1.3
Tuning required	No (RMSE below 1.5 threshold)

9.2 Streaming Results

Metric	Observed Value
Kafka events sent	2000 base + spikes (approx 2400 total)
Partitioning	cold_start to P0, power/regular to P1
Window size / slide	30 seconds / 10 seconds
Late events injected	5% of events (60-120 seconds late)
Late events handled	Dropped by watermark (older than 2 min)
Trending item alerts	Fired on items receiving 15 high ratings in spike
Activity spike alerts	Fired when user exceeds 8 interactions in 30s window
Recommendation latency	Measured per epoch in foreachBatch (see notebook output)

10. Challenges & Lessons Learned

10.1 Technical Challenges

- Docker disk space: The Spark image plus all Python packages (pyspark, JupyterLab, pandas, numpy, scikit-learn, scipy, matplotlib) required over 10GB of Docker virtual disk space. The default Docker disk limit of around 60GB filled up during the build process, causing OSError Errno 5 input/output errors on multiple attempts. Solution: docker system prune -a to clear cached layers, and increasing the Docker disk image size to 80GB in Docker Desktop settings.
- Java package availability: The openjdk-17-jdk package is not available in the python:3.9-slim Debian trixie image. Solution: switched to default-jdk-headless and dynamic JAVA_HOME=/usr/lib/jvm/default-java.
- WSL crashes during build: The build process required significant RAM for downloading and extracting the Spark tarball. Docker Desktop on Windows uses WSL2 which crashed when memory was insufficient. Solution: increased Docker memory allocation to 6GB in Docker Desktop Resources settings.
- ALS requires integer IDs: The Amazon dataset uses alphanumeric product IDs (e.g. B00XYZ123). ALS in Spark MLlib requires integer user and item columns. Solution: applied StringIndexer pipeline and cast to IntegerType, then saved the mapping tables as Parquet for lookup during streaming.
- Streaming and batch in same notebook: Running a long-running streaming query (Section 6) alongside the Kafka producer (Section 7) in a single Jupyter notebook required understanding that foreachBatch blocks the kernel. Solution: structured the notebook so Section 7 runs in a separate kernel execution context.

10.2 Lessons Learned

- Watermarking is essential for correctness: without withWatermark, Spark keeps state indefinitely for late data, causing memory growth. The 2-minute tolerance balances correctness against resource usage.
- Partition routing by user segment significantly improves streaming efficiency: keeping cold-start users on a dedicated partition prevents the ALS model lookup (which requires a broadcast join) from being triggered unnecessarily.
- The Bayesian average is more robust than simple average for popularity ranking: items with a single 5-star rating are smoothed toward the global mean, giving cold-start users more reliable recommendations.
- foreachBatch is the most flexible integration point between ML and streaming: it allows arbitrary Python code including model inference, unlike native Spark transformations which are restricted to DataFrame operations.
- Docker volume mounts (/app) provide a practical escape hatch for disk space issues: when the Docker virtual disk fills up, writing to the mounted host directory bypasses the Docker filesystem size limit entirely.

11. Run Instructions

Prerequisites

- Docker Desktop installed and running (minimum 6GB RAM, 80GB disk allocated)
- Git installed
- Windows 10/11 with WSL2 enabled

Steps

Clone the repository:

```
git clone https://github.com/Farahmostafa123/miniproject3.git
cd miniproject3
```

Start all containers:

```
docker-compose up --build -d
```

Open JupyterLab:

```
start http://localhost:8888
```

Open notebooks/main_notebook.ipynb and run Cells 1 through 6 in order
Run Cell 10 (Kafka producer) while Cell 9 streaming is active
Open dashboard/index.html in browser to view live dashboard

Service URLs

Service	URL
JupyterLab	http://localhost:8888
Spark Master UI	http://localhost:8080
Spark App UI	http://localhost:4040
Dashboard API	http://localhost:5050/api/health
Dashboard	Open dashboard/index.html in browser